# e-SITREP System — Written System Description

**Ministry of ICT and National Guidance (MoICT&NG)**
**Government Systems Prototype Showcase Submission**

---

## 1. Overview

The **Electronic Situation Report (e-SITREP)** system automates the compilation, verification, and analysis of daily border movement reports. Replacing error-prone manual processes, e-SITREP connects over 60 border posts to Headquarters (HQ) via a centralized, role-based workflow.

## 2. Component Breakdown

### Frontend or User Interface

- **Access Method**: Users access the system via modern web browsers (Chrome, Edge, Firefox). The interface is highly optimized for desktop environments to support the complex, spreadsheet-like data entry required at border stations, while remaining fully responsive.
- **Key Screens & Workflows**:
  - **Station Portal**: Border officers enter daily movements (arrivals, departures, asylum seekers) via dynamic grids. Air stations (e.g., Entebbe) access specialized modules for flights and deportees.
  - **HQ Inbox**: A workflow queue for HQ staff to review, verify, and approve submitted station reports or process data amendments.
  - **Reports & Outputs**: 1-click generation of the national Consolidated Daily SITREP and Weekly Excel matrix.

### Backend / Application Layer

- **Technology**: Built entirely on **Next.js 15 (App Router)** utilizing **TypeScript** and **Node.js**.
- **Processing Logic**: The application uses thin REST API handlers that delegate to isolated domain services ( `lib/*` ). These services handle complex business logic, such as report lifecycle

transitions (draft → submitted → reviewed → verified → approved), formatting consolidated strings, and compiling Excel/PDF exports.

## Database Layer

- **Technology**: **PostgreSQL 16**, an enterprise-grade relational database, interfaced via the **Prisma ORM**.
- **Data Stored**: Stores categorical movement data, border station metadata, user credentials, strict audit logs, and workflow states.
- **Hosting**: For the prototype showcase, the PostgreSQL database is securely hosted remotely on **Render**. The system is also designed to operate seamlessly with Docker for on-premise government servers if required.

## Authentication and Access

- **Authentication**: Secured via **NextAuth.js (v5)** using bcrypt password hashing and JSON Web Tokens (JWT) for session management. Multi-factor authentication (MFA) and Single Sign-On (SSO) integration (e.g., via Keycloak) are architecturally supported for Phase 2.
- **Access Control**: A stringent **Role-Based Access Control (RBAC)** engine enforces permissions at the middleware and API level. Roles include `STATION_INPUTTER`, `HQ_REVIEWER`, `HQ_VERIFIER`, `HQ_AUTHORISER`, and `ADMIN`. Users are strictly isolated to their assigned stations.

## APIs and Integrations

- **Exposed APIs**: e-SITREP exposes comprehensive RESTful JSON endpoints (`/api/*`) for all frontend operations, ensuring the core engine is decoupled and accessible.
- **Consumed Services**: Currently consumes the external `REST Countries API` to guarantee all nationality data adheres to ISO 3166-1 alpha-2 standards.
- **Integration Readiness**: The API-driven architecture allows for frictionless future integration with Uganda's Digital Public Infrastructure, such as **NIRA** (for identity verification) and **MIDAS/PISCES** (for automated border control data ingestion).

## Hosting and Infrastructure

- **Infrastructure**: The application frontend and API layer are currently deployed remotely on **Vercel** for high availability and serverless scalability, while the database is hosted on **Render**.
- **Deployment**: While the current prototype utilizes Vercel and Render for rapid iteration, the architecture is container-ready (`docker-compose`) and can be seamlessly migrated entirely on-premise within the MoICT/NCIC data centers in Uganda.

### Security Controls

- **In Transit & At Rest**: All data transmission is secured via HTTPS/TLS encryption. Sensitive data in the PostgreSQL database leverages native at-rest encryption on the host infrastructure.
- **Auditability**: An immutable, append-only `audit_logs` table captures every meaningful state change, recording the user ID, timestamp, target entity, and exact "before and after" JSON payloads.
- **Data Integrity**: Approved reports are cryptographically "locked". Revisions require an HQ-approved amendment workflow, ensuring historical data immutability.

### Data Flow

1. **Entry**: Border officers submit daily movement tallies through the React frontend.
2. **Transit**: JSON payloads are transmitted over HTTPS to the Next.js API layer.
3. **Validation**: The Next.js middleware verifies JWT sessions, RBAC permissions, and payload schemas.
4. **Persistence**: Domain logic processes the request, writes an audit log, and commits the transaction to PostgreSQL via Prisma.
5. **Extraction**: HQ staff trigger API requests to aggregate the persisted data into consolidated text or Excel format via the respective export engines.

---

# 3. Assessment for Government Deployment

## 3.1 Fitness for Government Deployment

e-SITREP is engineered for extreme reliability and scalability. The stateless Next.js architecture enables seamless horizontal scaling across multiple servers to handle thousands of concurrent users. PostgreSQL handles complex tabular aggregations efficiently. Critical single points of failure are mitigated through container orchestration and robust error boundaries.

## 3.2 Interoperability with Uganda's Digital Infrastructure

Integration is a foundational property of e-SITREP. By strictly enforcing the **ISO 3166-1 alpha-2 standard** for country codes across the entire database, the system ensures data normalization for future synchronization with the National Identification System (NIRA) or URA. The internal REST APIs are designed so they can easily be securely exposed to the Government integration layer via an API Gateway without requiring core rebuilds.

## 3.3 Security Design

Security is treated as an architectural cornerstone rather than a feature. Protection occurs at multiple perimeters: NextAuth intercepts unauthenticated requests, the RBAC engine blocks unauthorized API actions, and server-side validation strips malicious payloads. The immutable `audit_logs` table ensures total forensic accountability for every record alteration, satisfying stringent government compliance standards.

## 3.4 Technology Choices

The technology stack—TypeScript, React, Node.js, and PostgreSQL—was deliberately chosen for its non-proprietary nature, vast open-source community support, and avoidance of vendor lock-in. These skills are highly prevalent in Uganda's technical talent pool, ensuring that Ministry IT departments can seamlessly take over maintenance, support, and feature expansion without reliance on expensive, scarce specialists.

## 3.5 Future Procurement & Capability

This architecture proves capability beyond basic data entry; it demonstrates a deep understanding of enterprise workflows, data normalization, role-based security perimeters, and export generation. The principles applied here—strict auditing, decoupled API design, and ISO standards—are highly transferable to subsequent Government digitization initiatives in sectors like health, taxation, or social protection.